Name: . . . . . Number: . . . . . .

CSC 211—Fundamental Algorithms and Data Structures

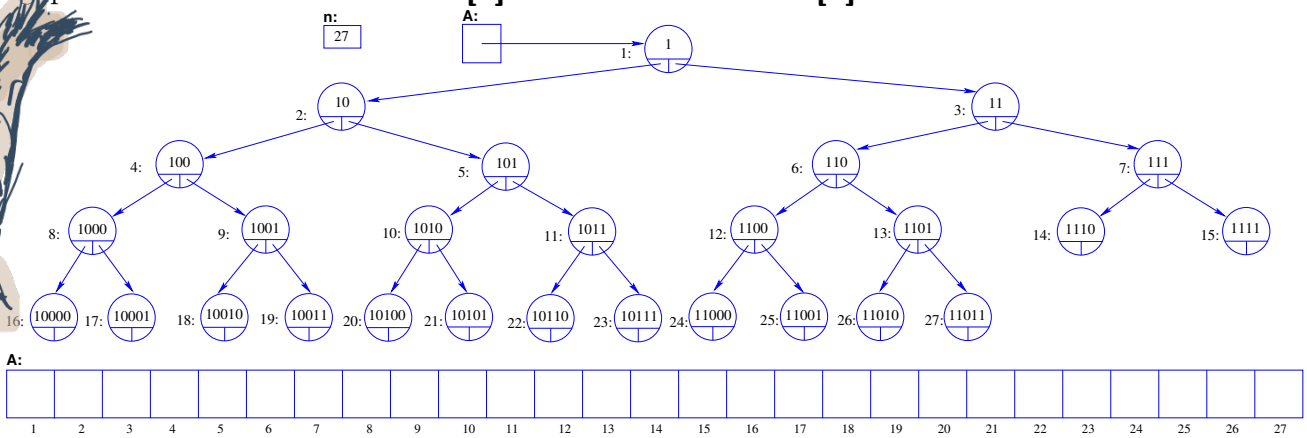
Heaps or priority queues

Term 1 Tutorial 6 — 18 March 2026

Submit today before leaving tutorial

Consult your tutors, Gemini, DeepSeek, ChatGPT, Claude, or Grok.

Heaps are the focus of today's tutorial—they are also known as priority queues. Heaps are based on complete trees, i.e., trees that are represented in an array of contiguous nodes. Every element of the tree for a heap is uniquely identified by considering its position as a binary number—this number reveals its *level*, *parent* or *children*, its left or right *arity* and also its precise *location* in the tree. A heap **A** is said to be *maximizing* when each of its keys has the property **A[i].key**≥**A[i//2].key**, where **//** indicates integer division. Another way of putting this, is that key of any parent is greater or equal to the key of any of its children. In a *min-heap* the inequality is ≤. Note that the subscript **i**≤**n**. When **i** exceeds **n** the "pointer" is **null**. Consider the complete tree structure **T** given in the figure. The arrows represent the *virtual links* between the nodes of a 27 node heap. The tree **T** is called *complete* when there are no gaps between the first node at **A[1]** and its last node at **A[n]**.



---

1. Consider the following heaps. Show the exact contents of each heap after removing the element with the "highest" priority OR Write NOT A HEAP. [15]

(a) A: 20 | 6 | 18 | 4 | 3 | 12 (1 2 3 4 5 6) A: 18 | 6 | 12 | 4 | 3 (1 2 3 4 5)

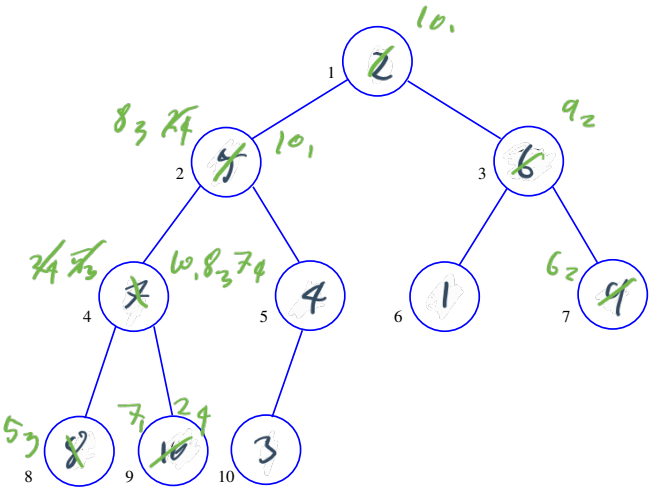
(b) A: 17 | 8 | 11 | 7 | 2 (1 2 3 4 5) A: 11 | 8 | 2 | 7 (1 2 3 4)

(c) A: 33 | 9 | 17 | 5 (1 2 3 4) A: 17 | 9 | 5 (1 2 3)

(d) A: 16 | 10 | 4 (1 2 3) A: 10 | 4 (1 2)

(e) A: 12 | 14 (1 2) A: 27 (1) (Minimising)

2. Build a MAXIMIZING *priority queue* or *heap* from the values 8, 7, 10, 2, 4, 6, 9, 3, 5, 1, that have already been loaded into the array used for the tree. The figure shows the initial data in the tree. THE ARRAY BELOW MUST BE FILLED IN AFTER YOU HAVE COMPLETED BUILDING THE HEAP FROM THE BOTTOM UP. If the heap were built strictly bottom-up, answer questions 3 and 4.



(a)

1	2	3	4	5	6	7	8	9	10
10	8	9	7	4	1	6	5	2	3

[8]

3. What is the *value* of the third node that has to be moved upward?

8 [2]

4. What is the *value* of the last node that has to be moved downward?

2 [2]

5. State whether each of the following lists IS or IS NOT a a heap—it IS a min- or max-heap or IS NOT a heap. Encircle IS or IS NOT, so that the sentence states whether the list is a heap or is not a heap AND encircle the first offending node when there is one. [16]

(a) 75, 71, 73, 70, 59, 72, 75, 70 IS | IS NOT a heap. (2)

(b) 127, 127, 129, 127, 127, 129, 130 IS | IS NOT a heap. (2)

(c) 62, 61, 63, 60, 59, 62, 61, 61 IS | IS NOT a heap. (2)

(d) 69, 61, 67, 61, 59, 66, 65, 61, 58, 57, 58, 65, 62, 61, 61, 60, 57, 56, 57, 56, 56, 55, 64, 63, 62, 61 IS | IS NOT a heap. (6)

(e) 87, 31, 61, 8, 16, 62, 59 IS | IS NOT a heap. (2)

(f) 87, 64, 32, 63, 61, 32, 32, 33 IS | IS NOT a heap. (2)

---

a) 2^{n-1}

b) $2^n - 1 - (2^{n-1}) = 2^n - 2^{n-1} - 1$

$2^n - 2^n \cdot 2^{-1} - 1$

$2^n(1 - \frac{1}{2}) - 1$

$2^n(\frac{1}{2}) - 1$

$\therefore 2^{n-1} - 1$

c) N (Always)

d) $2^{(n-1)} + 2^{(n-2)}$

$2^n(\frac{3}{4})$

$$2^{(n-1)} + 2^{(n-2)}$$

$$= 2^n \cdot 2^{-1} + 2^n \cdot 2^{-2}$$

$$= 2^n \left(1 \cdot \frac{1}{2} + 1 \cdot \frac{1}{4} \right)$$

$$= 2^n \left(\frac{3}{4} \right) = 2^{n-2} \times 3$$

$$\overbrace{\underbrace{11}_{3}\underbrace{00\ldots0}_{n-2}}^{n \text{ bits}}$$